

ENGIO

Engineering Intelligence Platform

Product Roadmap · All Phases

From first PR comment to full engineering intelligence

A 30-month phased build plan for market-ready SaaS

Confidential · Version 1.0 · March 2026

Executive Summary

EnginelQ is being built in six phases over 30 months, progressing from a focused PR review tool to a full engineering intelligence platform. Each phase delivers standalone commercial value while laying the foundation for the next. The sequencing is deliberate — start narrow, prove value on Billable, expand based on client pull rather than assumption.

The north star: a platform that instruments the entire software delivery lifecycle for South African engineering teams — from requirements through production — making engineering health measurable, governable, and continuously improving.

#	Phase	Core Capability	Timeline	Revenue Trigger
1	PR Review	Architecture-aware AI pull request analysis	Month 1–5	Starter / Growth
2	Architecture Intelligence	Continuous drift detection across the full codebase	Month 5–12	Growth add-on
3	Requirements Intelligence	Clarity scoring and acceptance criteria validation	Month 9–15	Growth add-on
4	Test Intelligence	AI-generated edge cases and test quality scoring	Month 12–18	Scale tier
5	Delivery Risk Intelligence	Sprint forecasting and board-level engineering visibility	Month 15–24	Executive tier
6	Runtime & Incident Intelligence	AI-assisted incident management and root cause analysis	Month 18–30	Enterprise tier

PRINCIPLE	Build what clients pull for. Billable is the validation environment throughout. Each phase only begins when the previous phase is generating real signal and real revenue.
-----------	--

Phase 1 — PR Review

Architecture-aware AI pull request analysis · Month 1–5

Overview

Phase 1 is the entry point into every client's engineering workflow. It delivers immediate, visible value — a PR gets reviewed within 90 seconds of opening, with findings that are architectural and meaningful, not just style nits. This is the land motion that earns trust before anything deeper is introduced.

The distinction that matters: most competitors are diff-aware (they read what changed). EngineIQ is system-aware (it understands how the change affects the architecture). That difference is the entire value proposition in Phase 1.

Milestones

M0	Working Loop	Weeks 1–2	First PR comment live
M1	Standards Engine	Weeks 3–5	Rule violations in PRs
M2	Context Builder	Weeks 6–8	Architecture-aware reviews
M3	Multi-Tenancy	Weeks 9–12	Two isolated tenants live
M4	Dashboard & Metrics	Weeks 13–16	CTO metrics report
M5	Production Hardening	Weeks 17–20	Platform sellable

Capabilities Delivered

Capability	Detail
GitHub App integration	Webhook-based PR event handling. HMAC-SHA256 validation. Async processing via Service Bus.

Architecture style detection	Automatic detection of Clean Architecture, Layered, Hexagonal, Modular Monolith from repo structure.
Standards Engine	YAML-configurable deterministic rules. Layer dependency checks, secret detection, async pattern enforcement, naming conventions.
AI Review Engine	Azure OpenAI GPT-4o with context-enriched prompts. Findings deduplicated against rule results. Zero raw code in persistence.
Severity classification	Critical / High / Medium / Suggestion. Grouped PR comment with developer-friendly recommendations.
Metrics dashboard	PR review history, violation trends, architecture drift score, 30-day email report.
POPIA compliance	All processing in Azure South Africa North. Ephemeral diff handling. DPA template for clients.
Multi-tenancy	Full tenant isolation at Service Bus, Redis, AI Search, and PostgreSQL row-level security.

Success Criteria

Metric	Target	Significance
PR review completion time	< 90 seconds	Developer experience threshold
False positive rate	< 15%	Above this, developers ignore the tool
Suggestion acceptance rate	> 40%	Indicator of finding quality
PR cycle time reduction (Billable)	> 30%	Headline ROI metric for CTO sales conversations
Paying clients by Month 5	3–5	Revenue validation before Phase 2 investment
Token cost per PR review	< R 8	Unit economics positive at R2,500/mo Starter tier

Revenue Model

Tier	Starter	Growth	Scale
Price / month	R 2,500	R 5,500	R 12,000
Repositories	Up to 3	Up to 10	Unlimited
PR Review	✓	✓	✓
Architecture Linter	—	✓	✓
Dashboard & Reports	Basic	Full	Full + API

SLA	Best effort	99.5%	99.9% + CSM
-----	-------------	-------	-------------

Phase 2 — Architecture Intelligence

Continuous codebase drift detection · Month 5–12

Overview

Phase 1 catches architectural violations in individual PRs. Phase 2 zooms out and asks a harder question: how is the overall health of this codebase changing over time? Architecture Intelligence produces a measurable drift score that CTOs can track, present to stakeholders, and use to justify engineering investment.

This is the upgrade trigger from Starter to Growth. When a client sees their drift score rising over 60 days, the conversation about the Architecture Linter module writes itself.

Capabilities Delivered

Capability	Detail
Full codebase scan	Scheduled scan of the entire repository, not just PR diffs. Detects accumulated drift between PRs.
Architecture Drift Score	A 0–100 score representing architectural health. Tracked over time. Drillable by layer, file, rule.
Cyclic dependency detection	Identifies circular dependencies between modules, namespaces, or layers that create hidden coupling.
Tight coupling alerts	Flags components with excessive afferent or efferent coupling that increase fragility and reduce testability.
Missing abstraction detection	Identifies where concrete implementations are used directly where interfaces should exist.
Drift threshold alerting	Configurable alert when drift score exceeds threshold. Email + webhook notification to CTO.
Trend reporting	Weekly drift trend chart in dashboard. Month-over-month architectural health comparison.

Technical Build

- Extend EngineIQ.ContextBuilder with full-repo traversal (not just PR-scoped diff).
- Build dependency graph analyser — maps namespace and type references across the solution.
- Implement coupling metrics: Afferent Coupling (Ca), Efferent Coupling (Ce), Instability ($I = Ce / Ca + Ce$).
- Add scheduled scan job to EngineIQ.Worker — runs nightly per active tenant.
- Extend EngineIQ.Dashboard with drift score timeline and drilldown views.

- Add alerting service — evaluates drift thresholds post-scan, dispatches notifications.

Commercial Trigger

GTM Run the full-codebase scan silently for Starter tier clients for 30 days. Surface the drift trend in a monthly email with a blurred detail section. The CTA is an upgrade to Growth to unlock the full Architecture Linter and drilldown. Conversion rate target: 30% of Starter clients within 90 days.

Success Criteria

Metric	Target
Drift score accuracy (validated against manual review)	> 80% agreement with senior engineer assessment
Starter → Growth upgrade rate	> 30% of Starter clients within 90 days of Phase 2 launch
Full-codebase scan runtime	< 10 minutes for repos up to 100K lines
CTO dashboard engagement (weekly active)	> 60% of Growth tier clients checking dashboard weekly

Phase 3 — Requirements Intelligence

Shift quality left — before a line is written · Month 9–15

Overview

Most production defects start at requirements. Ambiguous acceptance criteria, missing edge cases, contradictory business rules — these are the upstream causes of the downstream bugs that EngineIQ catches in PRs. Phase 3 moves the intervention point earlier: before development begins.

Requirements Intelligence analyses tickets and user stories at the point they are written, scores their clarity and completeness, and — critically — links them back to the eventual PR to verify the implementation matches the original intent.

Capabilities Delivered

Capability	Detail
Requirement clarity scoring	AI scores each ticket 0–100 on clarity, completeness, and testability before sprint planning.
Acceptance criteria checker	Flags missing given/when/then structure. Detects vague language ("should work", "as needed").
Contradiction detection	Identifies conflicting requirements within a sprint or epic using cross-ticket semantic analysis.
Edge case suggestions	Proactively suggests missing edge cases: timezone handling, error states, performance constraints, security classification.
Non-functional requirement prompts	Flags when performance, security, or availability NFRs are missing from a feature ticket.
Intent-to-implementation linking	When PR is opened, compares code changes against the linked ticket. Flags divergence between intent and implementation.
Integrations	Jira, Linear, Azure DevOps. Webhooks for ticket creation and sprint planning events.

Technical Build

- Build EngineIQ.RequirementsEngine — ticket ingestion, clarity scoring, edge case generation.
- Add Jira and Linear webhook listeners alongside the existing GitHub webhook listener.
- Build intent-to-implementation linker — extracts PR description, links to ticket ID, runs semantic similarity comparison.
- Extend PR comment with intent alignment section: "This PR implements X% of the linked ticket requirements."

- Add requirements health dashboard — clarity trends per sprint, team, and product area.

Why This Phase Changes the Sales Conversation

INSIGHT

Phase 1 and 2 sell to Engineering Managers and Tech Leads. Phase 3 sells to Product Managers and CTOs simultaneously. When EngineIQ starts evaluating tickets, it becomes relevant to the entire product delivery function — not just the engineering team. This broadens the buying coalition and increases switching cost significantly.

Phase 4 — Test Intelligence

Make testing semi-autonomous · Month 12–18

Overview

Coverage percentage is a vanity metric. 80% coverage can still miss every critical path. Phase 4 replaces coverage counting with coverage reasoning — understanding what is actually tested and what matters.

The primary capability is AI-generated edge case tests from PR diffs: when a developer opens a PR adding a new feature, EngineIQ generates the tests they probably missed. Developers accept or reject them directly from the PR comment.

Capabilities Delivered

Capability	Detail
Edge case test generation	Generates xUnit / NUnit tests for edge cases specific to the diff — null inputs, boundary values, error paths.
API contract test generation	For controller changes, generates contract tests verifying the API surface has not regressed.
Test quality scoring	Scores existing tests on assertion density, mock overuse, test isolation. Not just coverage percentage.
Flaky test detection	Identifies non-deterministic tests from CI run history. Flags and suggests root cause.
Untested critical path flagging	Cross-references new code against existing test suite. Flags business-critical logic with no test coverage.
Regression detection	Compares test results between PR base and head. Surfaces newly failing tests before merge.
Developer accept/reject flow	Generated tests appear as suggestions in the PR comment. Developer can accept into a new test file with one click.

Technical Build

- Build EngineIQ.TestEngine — test generation, quality scoring, flakiness detection.
- Integrate with CI/CD pipelines via GitHub Actions — read test results from workflow runs.
- Build test acceptance flow — generated tests posted as PR file suggestions, accepted via GitHub review API.
- Extend EngineIQ.Dashboard with test quality trends, flaky test registry, coverage reasoning view.

Success Criteria

Metric	Target
Generated test acceptance rate	> 35% of generated tests accepted by developers
Flaky test identification accuracy	> 85% of flagged tests confirmed flaky by team
Post-merge defect rate change	> 20% reduction in post-merge defects at 90 days
Scale tier conversion	> 25% of Growth clients upgrade to Scale for Test Intelligence

Phase 5 — Delivery Risk Intelligence

Board-level engineering visibility · Month 15–24

Overview

By Phase 5, EnginelQ has accumulated months of engineering data per client: PR review history, drift scores, test quality trends, and requirement clarity patterns. Delivery Risk Intelligence synthesises all of this into sprint-level and portfolio-level forecasting.

This is where EnginelQ stops being a developer tool and becomes an executive tool. The CTO can walk into a board meeting with a data-backed probability of sprint delivery, an architectural debt trajectory, and a change failure rate trend. No other SA-built tool is doing this.

Capabilities Delivered

Capability	Detail
Sprint failure probability	Using velocity trends, spillover history, story size distribution, and current sprint state to predict delivery risk.
Overcommitment detection	Flags sprints where team capacity vs committed story points creates statistically likely spillover.
Architecture debt growth rate	Projects the drift score trajectory at current merge rate. Estimates when technical debt becomes critical.
DORA metric tracking	Deployment frequency, lead time, change failure rate, MTTR — tracked and trended automatically.
Engineering health score	A composite 0–100 score combining drift, test quality, review depth, velocity stability, and defect rate.
Board-level report	Auto-generated monthly PDF report. Executive summary + engineering health metrics. Branded for the client.
Risk forecast API	REST API for clients to integrate EnginelQ data into their own BI tooling or project management dashboards.

Technical Build

- Build EnginelQ.DeliveryEngine — sprint data ingestion, forecasting model, DORA calculator.
- Integrate with Jira and Linear for sprint data (velocity, story points, spillover history).
- Build ML forecasting pipeline — train on accumulated client data to improve prediction accuracy.
- Build board-level PDF report generator — auto-scheduled, client-branded, emailed monthly.
- Expose EnginelQ metrics via REST API — enables client BI integrations.

Why This Is the Stickiest Module

RETENTION

When a CTO presents EngineIQ data to their board, the tool becomes institutionalised. It is no longer a developer tool someone can cancel — it is a governance tool the organisation depends on. Churn risk drops dramatically at this point. Target: < 5% annual churn for clients using Delivery Risk Intelligence.

Phase 6 — Runtime & Incident Intelligence

Close the loop — production back to requirements · Month 18–30

Overview

Phase 6 closes the loop. EnginelQ now has data from requirements (Phase 3), through architecture (Phase 2), PRs (Phase 1), tests (Phase 4), and delivery (Phase 5). Phase 6 connects this to what actually happens in production — incidents, errors, and runtime behaviour.

When a production incident occurs, EnginelQ can correlate it back to the PR that introduced the change, the architectural violation that was flagged but overridden, and the test that was never written. That correlation is the ultimate proof of value — and it closes the intelligence loop back to requirements.

Capabilities Delivered

Capability	Detail
Log and trace ingestion	Reads structured logs, distributed traces, and metrics from Azure Monitor, Datadog, or custom endpoints.
Deployment diff correlation	Correlates incidents with the most recent deployment. Links to the PR, diff, and findings that shipped.
Root cause hypothesis generation	AI analyses logs + deployment diff to generate ranked root cause hypotheses with supporting evidence.
Incident summarisation	Generates a human-readable incident summary from raw telemetry — severity, affected services, likely cause, blast radius.
Auto-drafted post-mortems	Produces a structured post-mortem draft from incident timeline, root cause, and remediation actions. Human reviews and publishes.
MTTR tracking	Tracks mean time to recovery per incident type, team, and service. Feeds into DORA metrics.
Findings-to-incident linkage	Retroactively links incidents to EnginelQ findings that were overridden or not actioned. Builds institutional learning.
Loop-close reporting	Monthly report showing the journey from requirement → PR finding → incident → fix. Quantifies the cost of ignored findings.

The Data Flywheel

By Phase 6, EnginelQ has accumulated something no global competitor can replicate for the SA market: months of engineering data across SA FinTech and HealthTech teams, including incident

patterns, architectural drift trajectories, delivery velocity benchmarks, and defect root causes specific to the SA development ecosystem.

MOAT

This data becomes EngineIQ's proprietary benchmark set. Clients can be told: "Your change failure rate is 2.3x the SA FinTech average for a team your size. Here is what high-performing SA teams do differently." That benchmark intelligence is the long-term defensible asset — no overseas competitor will ever have it.

Master Timeline

The following table maps every phase across a 30-month horizon. Phases overlap deliberately — Phase 2 begins building while Phase 1 is in production hardening. The overlap is where client feedback from Phase 1 informs Phase 2 priorities.

Phase	Q1 2026	Q2 2026	Q3 2026	Q4 2026	Q1–Q2 2027	2027+
Phase 1 — PR Review	BUILD ►	LIVE	SCALE	✓	✓	✓
Phase 2 — Architecture Intel		BUILD ►	LIVE	SCALE	✓	✓
Phase 3 — Requirements Intel			BUILD ►	LIVE	SCALE	✓
Phase 4 — Test Intelligence				BUILD ►	LIVE	SCALE
Phase 5 — Delivery Risk					BUILD ►	LIVE
Phase 6 — Runtime Intel						BUILD

Revenue Progression

Period	Clients	ARPU / mo	MRR Target	Phase Active
Month 1–5 (Phase 1)	1–5	R 3,500	R 17,500	Phase 1
Month 6–12 (Phase 2)	5–15	R 5,500	R 82,500	Phase 1–2
Month 12–18 (Phase 3–4)	15–30	R 7,500	R 225,000	Phase 1–4
Month 18–24 (Phase 5)	30–60	R 9,500	R 570,000	Phase 1–5
Month 24–30 (Phase 6)	60–100	R 12,000	R 1,200,000	Phase 1–6

Risk Register & Governance

Risk	Severity	Mitigation	Owner
False positive rate erodes developer trust	HIGH	Configurable rules + developer override mechanism. Target < 15%. Monitored weekly.	Product + Engineering
Azure OpenAI latency spikes break 90s SLA	MEDIUM	Async processing decouples webhook from review. Dead-letter queue for retries. Chunking for large PRs.	Engineering
IP sensitivity prevents trial adoption	HIGH	POPIA compliance, ephemeral processing, DPA template, Azure SA North residency. Lead with these in sales.	Sales + Legal
Scope creep delays Phase 1 delivery	HIGH	Strict milestone gating. Phase 2 does not begin until Phase 1 has paying clients.	Founder
Global competitor enters SA market	MEDIUM	POPIA compliance and rand pricing are structural advantages. SA-specific benchmark data builds moat over time.	Product
Azure SA North capacity constraints	LOW	Monitor capacity limits early. Container Apps scale horizontally. Service Bus absorbs spikes.	Engineering
Over-reliance on Azure OpenAI pricing	MEDIUM	Deterministic rules handle high-volume low-cost checks. AI reserved for complex analysis. Token budget per PR enforced.	Engineering

Phase Gate Criteria

No phase begins until the previous phase has met its gate criteria. This prevents building complexity on an unvalidated foundation.

Gate	Criteria to begin next phase
Phase 1 → 2	3+ paying clients active. False positive rate < 15% sustained for 30 days. PR review SLA met consistently.
Phase 2 → 3	At least 1 client upgraded from Starter to Growth using architecture drift report as trigger. Drift score validated by client tech lead.
Phase 3 → 4	Requirements Intelligence integrated with at least 2 clients on Jira or Linear. Intent-to-implementation accuracy > 70%.
Phase 4 → 5	Generated test acceptance rate > 35% sustained. Scale tier has at least 2 paying clients. Post-merge defect rate measurably declining.

Phase 5 → 6	Sprint failure prediction accuracy > 70%. At least 1 client presenting EngineIQ data to their board. MRR > R 500,000.
-------------	---

The Endgame

By the end of Phase 6, EngineIQ is the engineering nervous system for the companies it serves. It knows their requirements, their architecture, their tests, their delivery patterns, and their production incidents — and it connects all of those together into a continuous intelligence loop that makes every engineering decision better informed.

The platform has built something no global competitor can quickly replicate: a rich dataset of SA engineering patterns, benchmarks, and incident histories specific to FinTech, HealthTech, and SaaS teams operating in the South African context. That dataset is the long-term moat.

**"AI-Augmented Engineering is not about replacing developers.
It is about reducing entropy in complex systems.
That's an architect-level conversation."**

EngineIQ starts that conversation with every CTO it meets. The roadmap above is how it earns the right to keep having it.